

Ingeniería en Sistemas de Información y Ciencias de la Computación

Autómatas y Lenguajes Formales

Gerson Samuel
Guzmán Franco **3490-24-14978**

RoboScript: Documentación técnica del analizador léxico y sintáctico

Asignatura: Autómatas y Lenguajes Formales Herramientas: Java, JFlex 1.9.1, CUP 0.11b, IntelliJ IDEA

1. Descripción general del lenguaje

RoboScript es un lenguaje formal de dominio específico (DSL) diseñado para describir el comportamiento de un robot simulado. Un programa RoboScript se compone de un encabezado que identifica al robot y su posición inicial, seguido de una secuencia no vacía de instrucciones. Las instrucciones pueden ser simples (movimiento, giro, manipulación de objetos, control de velocidad) o estructuras de control (repetición y decisión según la presencia de un obstáculo).

El proyecto implementa un procesador en dos fases, conforme al modelo clásico de construcción de compiladores:

1. **Análisis léxico** (JFlex): transforma el flujo de caracteres en una secuencia de tokens, ignorando espacios y saltos de línea, y registrando línea y columna de cada token.
2. **Análisis sintáctico** (CUP): verifica que la secuencia de tokens pertenezca al lenguaje generado por la gramática y acumula estadísticas mediante acciones semánticas.

Las expresiones regulares se definen formalmente en el archivo Scanner.flex; el código Java no utiliza Pattern, Matcher ni String.matches para validar el contenido.

2. Especificación léxica

2.1 Palabras reservadas

Todas las palabras reservadas distinguen mayúsculas y minúsculas y se escriben en mayúsculas.

Token	Lexema	Uso en la gramática
ROBOT	ROBOT	Inicio del encabezado
INICIO	INICIO	Separa el código del robot de su posición inicial
AVANZAR	AVANZAR	Movimiento hacia adelante
RETROCEDER	RETROCEDER	Movimiento hacia atrás
GIRAR	GIRAR	Giro (requiere dirección)
DERECHA	DERECHA	Dirección de giro
IZQUIERDA	IZQUIERDA	Dirección de giro
SEGUIR	SEGUIR	Primera parte de SEGUIR LINEA
LINEA	LINEA	Segunda parte de SEGUIR LINEA
LEVANTAR	LEVANTAR	Toma un objeto
MOVER	MOVER	Desplazamiento a una zona o coordenada
SOLTAR	SOLTAR	Libera el objeto sostenido
REPETIR	REPETIR	Inicio de bloque repetitivo

SI	SI	Inicio de bloque condicional
SINO	SINO	Rama alternativa del condicional
FIN	FIN	Cierre de bloque REPETIR o SI
OBSTACULO	OBSTACULO	Condición del bloque SI
VELOCIDAD	VELOCIDAD	Cambio de velocidad

2.2 Tokens definidos por expresión regular

Token	Expresión regular (JFlex)	Ejemplos válidos	Ejemplos inválidos
T_CODIGO_ROBOT	[0-9]{3}[A-Z]{3}	123ROB, 456BOT	12ROB, 123rob, 1234AB
T_CODIGO_ZONA	"Z-"[0-9]{2}	Z-05, Z-12	Z-5, Z05, Z-123
T_CODIGO_OBJETO	"OBJ-"[0-9]{4}	OBJ-0025, OBJ-0100	OBJ-25, OBJ0025
T_COORDENADA	"X"[0-9]+","[\t\r\n\f]*"Y"[0-9]+	X10,Y05, X20, Y15	X10 Y05, X,Y5
T_PORCENTAJE	("100"|[0-9]{1,2})"%"	75%, 100%, 0%	101%, 150%, 75
T_DURACION	[0-9]+[sm]	30s, 1m	30h, s30
T_NUMERO	[0-9]+	3, 10, 007	-5, 3.5

Nota: en JFlex la clase predefinida \s se expresa explícitamente como [\t\r\n\f], de modo que la coordenada admite espacios o saltos de línea entre la coma y la letra Y.

2.3 Elementos ignorados y control de posición

Se ignoran espacios, tabulaciones y saltos de línea mediante la regla [\t\r\n\f]+. JFlex mantiene los contadores internos yyline y yycolumn (base 0) mediante las directivas %line y %column; el analizador los incrementa en 1 para presentar posiciones base 1 y los almacena en cada símbolo (campos left = línea y right = columna).

2.4 Resolución de ambigüedades léxicas

JFlex aplica dos criterios, que el diseño aprovecha de forma deliberada:

1. **Coincidencia más larga**: ante 123ROB, se elige T_CODIGO_ROBOT (6 caracteres) y no T_NUMERO (123); ante 30s se elige T_DURACION y no T_NUMERO.
2. **Prioridad por orden de declaración** cuando la longitud empata: las palabras reservadas se declaran antes que los patrones genéricos, de modo que SI se reconoce como palabra reservada.

2.5 Errores léxicos

Se definen dos reglas finales de captura:

- [A-Za-z0-9_\-]+ (secuencia inválida): agrupa en un único error una palabra o código mal formado, por ejemplo 12ROB, OBJ-25 o avanzar. Al ser más larga o igual que cualquier otro patrón, solo se activa cuando ninguna regla válida cubre toda la secuencia.
- [^] (cualquier carácter): reporta un símbolo aislado no reconocido, como @.

Cada error registra línea, columna y el texto no reconocido. El analizador léxico continúa tras el error para informar todos los errores léxicos del archivo en una sola ejecución.

3. Gramática formal

3.1 Notación BNF (la implementada en CUP)

<programa> ::= <encabezado> <instrucciones>

<encabezado> ::= ROBOT T_CODIGO_ROBOT INICIO T_COORDENADA

<instrucciones> ::= <instrucciones> <instruccion>

| <instruccion>

<instruccion> ::= <simple> | <control>

<simple> ::= AVANZAR T_NUMERO

| AVANZAR T_DURACION

| RETROCEDER T_NUMERO

| RETROCEDER T_DURACION

| GIRAR DERECHA

| GIRAR IZQUIERDA

| SEGUIR LINEA

| SEGUIR LINEA T_DURACION

| LEVANTAR T_CODIGO_OBJETO

| MOVER T_CODIGO_ZONA

| MOVER T_COORDENADA

| SOLTAR

| SOLTAR T_CODIGO_ZONA

| VELOCIDAD T_PORCENTAJE

<control> ::= REPETIR T_NUMERO <instrucciones> FIN

| SI OBSTACULO <instrucciones> FIN

| SI OBSTACULO <instrucciones> SINO <instrucciones> FIN

3.2 Notación EBNF equivalente

programa = encabezado , instruccion , { instruccion } ;

encabezado = "ROBOT" , T_CODIGO_ROBOT , "INICIO" , T_COORDENADA ;

instruccion = simple | control ;

simple = ("AVANZAR" | "RETROCEDER") , (T_NUMERO | T_DURACION)

| "GIRAR" , ("DERECHA" | "IZQUIERDA")

| "SEGUIR" , "LINEA" , [T_DURACION]

| "LEVANTAR" , T_CODIGO_OBJETO

| "MOVER" , (T_CODIGO_ZONA | T_COORDENADA)

| "SOLTAR" , [T_CODIGO_ZONA]

| "VELOCIDAD" , T_PORCENTAJE ;

control = "REPETIR" , T_NUMERO , cuerpo , "FIN"

| "SI" , "OBSTACULO" , cuerpo , ["SINO" , cuerpo] , "FIN" ;

cuerpo = instruccion , { instruccion } ;

3.3 Decisiones de diseño

- Los cuerpos de bloque exigen al menos una instrucción, lo que evita bloques vacíos.
- La gramática es **LALR(1)** y no presenta conflictos: el problema del *else* ambiguo no aparece porque todo bloque SI se cierra con su propio FIN, lo que delimita sin ambigüedad el alcance de SINO.
- La recursión izquierda en instrucciones es la forma natural para un analizador ascendente y usa pila de tamaño constante para listas largas.
- Las instrucciones opcionales (por ejemplo SOLTAR con o sin zona) no generan conflicto porque T_CODIGO_ZONA no puede iniciar una instrucción.

4. Funcionamiento del analizador léxico (JFlex)

El archivo Scanner.flex se organiza en las tres secciones estándar de JFlex:

1. **Código de usuario**: declaración del paquete roboscript e importaciones.
2. **Opciones y definiciones**: %class Lexer, %unicode, %line, %column, %cup (compatibilidad con CUP), código auxiliar entre %{ ... %} y las macros de las expresiones regulares.
3. **Reglas léxicas**: cada regla asocia un patrón con una acción Java que devuelve un java_cup.runtime.Symbol.

JFlex compila las expresiones regulares a un autómata finito determinista (AFD) minimizado, que se genera en la clase Lexer.java. Cada invocación a next_token() recorre el AFD desde el estado inicial, recuerda el último estado de aceptación alcanzado y devuelve el lexema más largo (principio de *maximal munch*).

El método auxiliar tok() incrementa el contador de tokens, registra la información del token (categoría, lexema, línea, columna) y construye el símbolo. El método error() almacena los errores léxicos sin interrumpir el análisis.

5. Funcionamiento del analizador sintáctico (CUP)

El archivo Parser.cup declara los terminales (los mismos nombres que el analizador léxico), los no terminales, el símbolo inicial y las producciones. CUP construye las tablas de acción y de salto de un analizador **LALR(1)** y genera Parser.java y sym.java (constantes numéricas de los terminales).

Acciones semánticas. Las acciones {: ... :} se ejecutan en cada reducción. Incrementan los contadores de la clase Estadisticas (objeto accesible como parser.stats):

Contador	**Se incrementa con**
Movimientos	AVANZAR, RETROCEDER, MOVER, SEGUIR LINEA
Giros	GIRAR DERECHA, GIRAR IZQUIERDA

Objetos levantados	LEVANTAR
Objetos soltados	SOLTAR
Cambios de velocidad	VELOCIDAD
Bloques	REPETIR, SI (y se cuenta aparte cuántos SI tienen SINO)
Total de instrucciones	instrucciones simples + bloques

Las estadísticas son **estáticas**: cuentan las apariciones en el código fuente y no las ejecuciones (un bloque REPETIR 3 con un AVANZAR cuenta un único AVANZAR).

Integración. La clase Main ejecuta primero una pasada completa del analizador léxico (para reportar todos los errores léxicos) y, solo si no los hay, crea un nuevo Lexer y lo entrega a Parser, que consume los tokens bajo demanda.

6. Manejo de errores

6.1 Errores léxicos

Formato: Error léxico en línea L, columna C: <descripción> '<texto>'. Se reportan todos los errores del archivo y, si existe alguno, no se ejecuta la fase sintáctica, ya que un token omitido produciría errores sintácticos en cascada que no reflejan la causa real.

6.2 Errores sintácticos

Se redefine el método syntax_error de CUP. Formato: Error sintáctico en línea L, columna C: se encontró <token>; se esperaba: <lista>. La lista de tokens esperados se obtiene con expected_token_ids(), que consulta la tabla de acciones del estado actual del autómata. El análisis se detiene en el primer error (el método unrecovered_syntax_error se redefine para no lanzar excepciones).

Observación importante: un analizador LR detecta el error en el **primer token que no puede continuar ningún prefijo válido**, que no siempre está en la misma línea que la causa. Por ejemplo, en AVANZAR seguido de un salto de línea y GIRAR, el error se informa en la línea de GIRAR.

Bloques sin FIN. Si falta un FIN, el analizador llega al fin de archivo en un estado que espera una instrucción o FIN. El mensaje indica se encontró fin de archivo y lista FIN entre los tokens esperados.

7. Ejemplos de archivos de prueba

7.1 Archivo válido 1: flujo normal (valido1_flujo_normal.robot)

ROBOT 123ROB INICIO X10,Y05

VELOCIDAD 75%

AVANZAR 10

GIRAR DERECHA

AVANZAR 5

SEGUIR LINEA 30s

LEVANTAR OBJ-0025

MOVER Z-05

GIRAR IZQUIERDA

RETROCEDER 3

MOVER X20, Y15

SOLTAR Z-05

Explicación. El encabezado usa T_CODIGO_ROBOT y T_COORDENADA. Se emplean todos los tipos de token: porcentaje (75%), duración (30s), objeto, zona y una coordenada con espacio tras la coma (X20, Y15).

Resultado esperado: archivo válido con las siguientes estadísticas.

Métrica	**Valor**
Tokens reconocidos	27
Total de instrucciones	11
Movimientos	6 (AVANZAR 2, RETROCEDER 1, MOVER 2, SEGUIR LINEA 1)
Giros	2
Objetos levantados	1
Objetos soltados	1
Cambios de velocidad	1
Bloques	0

7.2 Archivo válido 2: estructuras anidadas (valido2_estructuras_anidadas.robot)

```
ROBOT 456BOT INICIO X0,Y0
VELOCIDAD 100%
REPETIR 3
  AVANZAR 2
  SI OBSTACULO
    GIRAR DERECHA
    REPETIR 2
      AVANZAR 1m
      SI OBSTACULO
        RETROCEDER 1
      FIN
    FIN
  SINO
    SEGUIR LINEA
    LEVANTAR OBJ-0100
  FIN
  GIRAR IZQUIERDA
```

FIN

MOVER Z-12

SOLTAR

Explicación. Un REPETIR contiene un SI/SINO, y la rama SI contiene a su vez un REPETIR que encierra otro SI. Cada bloque se cierra con su FIN correspondiente; el SINO pertenece al SI más externo porque el SI interno ya se cerró con su FIN.

Resultado esperado: archivo válido.

Métrica	**Valor**
Tokens reconocidos	36
Total de instrucciones	14 (10 simples + 4 bloques)
Movimientos	5
Giros	2
Objetos levantados / soltados	1 / 1
Bloques	4 (REPETIR 2, SI 2, con SINO 1)

7.3 Archivo inválido 1: error léxico (invalido1_lexico.robot)

ROBOT 12ROB INICIO X10,Y05

AVANZAR 10

GIRAR DERECHA @

LEVANTAR OBJ-25

SOLTAR Z-05

Salida esperada:

Error léxico en línea 1, columna 7: secuencia no reconocida '12ROB'

Error léxico en línea 3, columna 15: símbolo no reconocido '@'

Error léxico en línea 4, columna 10: secuencia no reconocida 'OBJ-25'

RESULTADO: ARCHIVO INVÁLIDO (3 error(es) léxico(s)).

Explicación. 12ROB tiene solo dos dígitos, por lo que no cumple [0-9]{3}[A-Z]{3}; @ no pertenece al alfabeto del lenguaje; OBJ-25 tiene dos dígitos en lugar de cuatro. Los tres errores se informan en una sola ejecución.

7.4 Archivo inválido 2: error sintáctico (invalido2_sintactico.robot)

ROBOT 789ABC INICIO X5,Y5

VELOCIDAD 50%

AVANZAR

GIRAR DERECHA

Salida esperada:

Error sintáctico en línea 4, columna 1: se encontró 'GIRAR'; se esperaba: T_NUMERO, T_DURACION

Explicación. Léxicamente el archivo es correcto, pero AVANZAR exige un argumento numérico o una duración. El error se detecta en el token GIRAR (línea 4), el primero que no puede seguir a AVANZAR.

7.5 Archivo inválido 3 (adicional): bloques sin FIN (invalido3_sin_fin.robot)

```
ROBOT 789ABC INICIO X5,Y5

REPETIR 3

  AVANZAR 2

  SI OBSTACULO

    GIRAR DERECHA

  SINO

    RETROCEDER 1
```

Salida esperada: Error sintáctico en línea 8, columna 1: se encontró fin de archivo; se esperaba: ...FIN... (la lista incluye FIN y los tokens que inician una instrucción; la línea corresponde al fin de archivo si el archivo termina con salto de línea).

Explicación. Los bloques SI y REPETIR están abiertos y el archivo termina antes de encontrar sus FIN. Un recorrido ascendente puede apilar cualquier cantidad de bloques abiertos, y solo al llegar al fin de archivo se detecta que faltan cierres.

8. Compilación y ejecución

1. Colocar jflex-1.9.1.jar, java-cup-11b.jar y java-cup-11b-runtime.jar en la carpeta lib/.
2. Generar el analizador léxico: java -jar lib/jflex-1.9.1.jar -d src/roboscript spec/Scanner.flex
3. Generar el analizador sintáctico: java -jar lib/java-cup-11b.jar -package roboscript -parser Parser -destdir src/roboscript spec/Parser.cup
4. Compilar: javac -encoding UTF-8 -cp lib/java-cup-11b-runtime.jar -d out src/roboscript/*.java
5. Ejecutar: java -cp out:lib/java-cup-11b-runtime.jar roboscript.Main tests/valido1_flujo_normal.robot (en Windows, usar ; como separador del classpath). La opción --tokens muestra la tabla de tokens reconocidos.

9. Conclusiones

El proyecto demuestra la aplicación práctica de la teoría de lenguajes formales: los tokens de RoboScript son un lenguaje regular, reconocido por un autómata finito determinista generado por JFlex, mientras que la estructura de bloques anidados con cierre obligatorio es un lenguaje libre de contexto, que requiere la pila de un autómata de pila determinista, implementado por el analizador LALR(1) de CUP. Esta separación justifica el uso de dos herramientas especializadas en lugar de expresiones regulares aplicadas al archivo completo, ya que un lenguaje con bloques anidados arbitrariamente no puede describirse con expresiones regulares.